课办 StudyFlow

AI 作业协作与规划助手

系统分析与设计 • Assignment 1 / Part I

项目构想提交（Ideate）

第24组 · 计算机科学与技术学院

董泽福 2451201 / 雍致鸣 2452016 / 李一芃 2454326

指导教师：刘岩

2026 年 10 月 11 日

提交说明

本文件整合项目构想、业务流程图、产品架构图与 AI 工具使用反思。文中指标、预算和开发排期均为拟议目标，并非实测结果。

项目构想：2026/10/12 13:30 前提交；完整 SRS 与建模文件：2026/10/23 19:00 提交。12 周计划为团队初步安排，后续按课程要求调整。

1. 项目封面、团队信息与项目简介

1.1 项目封面与团队信息

项目	内容
课程名称	系统分析与设计
报告名称	课办StudyFlow——AI 作业协作与规划助手方案设计报告
中文标题	AI 作业协作与规划助手
英文名称	StudyFlow
小组	第24组
学院	计算机科学与技术学院
指导教师	刘岩
项目方向	教育学习、生产力与协作、AI 智能代理
一句话定位	把课程作业要求转化为可协作、可调整、可检查的学习执行计划。

成员	姓名	学号	主要职责
A	董泽福	2451201	需求分析、交互原型与前端整合
B	雍致鸣	2452016	后端、数据库、权限与计划版本
C	李一芃	2454326	AI 代理、要求抽取与排程算法

1.2 项目简介

课办面向同时处理多门课程任务的大学生及课程作业小组，将作业说明转化为可追溯的要求清单、协作任务和可执行排期。智能代理根据成员可用时间、任务依赖和截止日期提出规划建议，通过业务工具检查约束，再由用户确认生效。项目重点研究需求追溯、多人协作、约束排程、计划版本和人机协同。

系统以响应式 Web 应用交付，完成“录入作业—确认要求—拆解分工—安排时间—协作执行—检查交付”的完整流程。首版按 12 周安排开发，优先完成可验证的课程作业管理流程，后续再扩展文件导入和日历集成。

2. 问题与动机：背景、主要目标与范围

2.1 真实问题与研究动机

大学课程作业往往同时包含报告、代码、展示和过程材料。要求可能散落在教学平台、课件和群消息中；学生即使记下截止日期，也不一定清楚要完成哪些交付物、谁负责、哪些任务必须先做，以及如何与其他课程安排协调。

小组作业进一步引入成员时间不一致、任务依赖和进度信息不透明等问题。例如，报告作者需要等待实验结果，展示负责人需要等待报告结构确定。若只是分别建立待办事项，任务之间的阻塞关系容易被忽略，临近截止时才暴露延期风险。

本项目拟研究三个问题：如何将自然语言要求转换为能够核对来源的结构化需求；如何结合个人时间与小组依赖生成可行计划；如何在作业要求或成员安排变化时，解释影响并协商调整。

前期调研计划：第 1–2 周访谈 6–8 名学生，其中至少包含 2 名小组负责人；在获得同意后收集 3–5 份脱敏作业说明。重点核查“漏要求、协调困难、延期”是否存在以及出现在哪些环节。使用现有表格或待办工具完成同一规划任务，记录耗时作为基线。若排程需求较弱，则减少排程深度，优先完善要求追溯和小组进度管理。

2.2 用户痛点及影响

以下为需求分析阶段拟验证的痛点，访谈后根据实际频率与影响调整优先级。

痛点	典型表现	对用户的影响	对应解决方向
作业要求分散，难以核对完整性	截止日期、格式和交付物出现在不同材料中	遗漏要求，提交前集中返工	抽取要求并保存原文出处，形成可逐项核对的清单（A-F02、A-F08）
分工只列负责人，未说明依赖	写报告的人等待实验结果，制作展示的人等待内容定稿	成员互相等待，关键任务延误	显式记录任务前置关系、负责人和阻塞原因（A-F03、A-F06）
多门课程与成员时间相互冲突	各自制定的计划占用同一时段，小组难以协调	计划无法执行，临近截止集中赶工	将个人可用时间与小组任务共同纳入约束排程（A-F04）
计划变更缺少统一记录	群聊中调整分工，但任务清单未同步	成员执行不同版本的计划，责任不清	展示计划差异，经确认后形成新版本并通知成员（A-F07）
任务完成不等于交付要求已覆盖	看板上任务都已完成，仍缺少某项附件或展示材料	提交前缺乏可靠的完整性检查	建立要求、任务和交付证据之间的关联（A-F08）

2.3 主要目标

目标	设计响应	预期验证方式
降低遗漏作业要求的可能性	每条要求关联原文、任务及交付证据	比较人工标注与系统抽取结果；检查遗漏清单
降低首次制定计划的成本	自动提出任务拆解、工时估计与时间安排	与人工规划相同案例的耗时比较
提高小组协调透明度	明确负责人、依赖、阻塞原因与验收状态	邀请成员完成一次模拟小组协作
支持可控的动态调整	展示新旧计划差异、影响对象及原因	验证延期、请假、截止日期变更场景
形成可分析的系统设计	建立权限、状态机、数据模型和可追溯需求	需求与用例、设计、测试逐项关联

2.4 系统范围

首版范围（P0，必须完成）：账号与小组空间、课程和作业管理、粘贴文本导入、要求抽取与人工确认、任务及依赖管理、可用时间录入、排程建议、计划确认与版本记录、任务看板、交付证据链接、站内提醒。

扩展范围（P1，首版验收后再做）：文本型 PDF 导入、日历文件导出、历史工时辅助估计、教师或助教只读视图。扫描件 OCR、多校教学平台集成、在线文档实时合编留作后续研究。

范围边界：系统帮助理解与组织作业，不自动代写完整作业、不替学生提交到学校平台、不自动判定学术成绩。首版使用学生主动录入的作业和可用时段，不依赖学校开放接口。个人时间安排仅向小组展示必要的忙闲信息。

3. 解决方案：功能、关键特性、用户与可用性

3.1 目标用户与角色

首版主要服务高校学生，优先关注需要同时提交报告、程序、展示等成果的课程作业小组，同时支持个人管理多门课程任务。教师和助教属于后续只读视图的潜在用户，不是首版必须支持的操作角色。

角色	主要诉求	权限边界
学生 / 小组成员	知道自己的任务、截止日期与阻塞原因	维护本人可用时间，更新本人任务，查看所属小组共享内容
小组负责人	分配任务、协调依赖、确认整体计划	管理成员和作业，提出分工，确认小组计划及交付验收
个人作业所有者	管理独立完成的课程任务	管理本人个人空间；个人数据不自动向小组开放

同一学生可在不同小组承担不同角色。权限按空间判断，不因为在一个小组担任负责人就获得另一个小组的数据访问权。

3.2 典型业务场景与业务流程示意图

四名学生需要在两周后提交某课程的调研报告、展示材料与演示程序。负责人粘贴作业说明，系统抽取截止时间、交付物和明确约束，并展示每条信息的原文位置。负责人确认后，AI 提出资料调研、需求分析、设计、实现、整合与演练等任务，标明哪些来自明确要求、哪些是规划建议。

成员填写可用时段和预计工时。排程服务根据依赖与可用时间生成计划；若时间不足，系统列出无法安排的任务与冲突原因。负责人可以修改工时、分工或时间窗口，然后确认版本。某成员临时减少可用时间时，系统生成受影响任务和调整建议，经确认后更新计划。提交前，要求检查页集中展示尚未关联交付证据的项目。

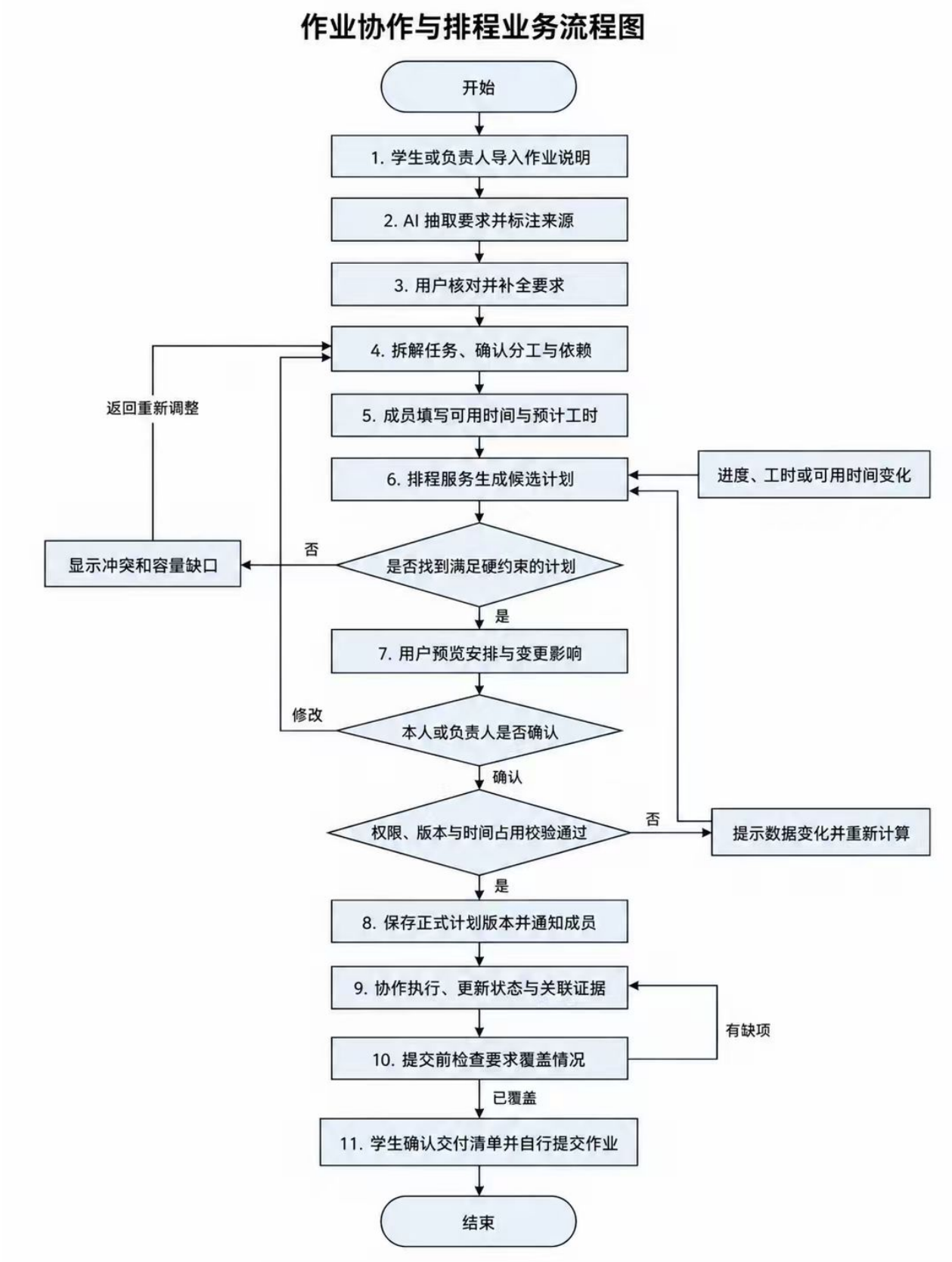


图 1：作业协作与排程业务流程

图中展示正常完成路径，以及排程冲突、用户修改、确认时数据过期和提交前发现缺项的处理路径。个人作业由本人确认，小组作业由负责人确认；最终提交仍由学生在学校指定渠道完成。

3.3 核心功能需求

编号	功能	关键行为与验收要点
A-F01	空间、课程与作业管理	支持个人和小组作业；后端拒绝跨空间访问；记录明确的截止日期与时区
A-F02	作业要求结构化	从文本抽取交付物、截止日期、格式要求；每项显示来源；不确定信息待人工补全，确认前不成为正式要求
A-F03	任务拆解与依赖	设置负责人、预计工时、优先级及前置任务；阻止循环依赖；AI 估计必须可修改
A-F04	约束排程	按截止日期、依赖和可用时段安排任务；不能满足时明确返回冲突，不能悄悄放宽硬约束
A-F05	规划智能代理	根据用户问题调用查询和排程工具；生成结构化计划草案及原因说明；确认后才应用计划
A-F06	小组协作看板	记录任务状态、评论和阻塞原因；提交成果后由负责人验收；任务状态变化可追踪
A-F07	变更与版本管理	预览调整前后差异；保存确认人、时间和依据版本；旧草案不得覆盖期间发生的新修改
A-F08	完成检查与提醒	将要求关联到任务及成果链接，列出未覆盖项；站内展示临期、逾期和依赖阻塞提醒

首版以评论、链接及状态更新实现协作，不承担在线文档编辑器的实现工作。要求“已覆盖”表示已有对应交付证据，不等于内容已达到评分标准。

3.4 关键业务规则与验收场景

1. **信息来源规则**：明确要求、用户补充和 AI 建议分别标识。原文没有截止日期时不得自行生成一个“官方截止日期”。

2. **排程硬约束**：同一学生已确认的时段不能重叠；任务只能进入其可用时段；后继任务的计划开始不能早于前置任务的计划结束；受截止日期约束的交付工作不能排到截止日期之后。

3. **偏好与不可行性**：尽量均衡每日负担、减少切换属于优化偏好。当总工时超过可用容量时，显示缺口，由用户调整投入、任务范围或分工，不伪造可行结果。

4. **执行状态规则**：任务采用“待开始→进行中→待验收→已完成”；验收退回时回到进行中。阻塞作为单独标记及原因保存。前置任务实际未完成时，后继任务显示阻塞，即使原定时间已经到达。

5. **计划确认规则**：个人计划由本人确认，小组计划由负责人确认；受影响成员收到站内通知。成员可用时间或预计工时变化后，旧安排标记为待重排，系统不无声移动其他人的任务。

6. **并发规则**：草案绑定作业、任务和可用时间的版本。确认时发现版本变化或新的时间占用，则拒绝直接应用并提示重新计算；对多个成员的时间占用检查与计划写入在同一事务内完成。

代表性验收案例：

缺失信息：输入不含日期的说明，结果应显示“截止日期待确认”，仍可保存草稿。

循环依赖：先设置任务甲依赖乙，再设置乙依赖甲，应拒绝第二次操作并指出环路。

跨作业冲突：个人作业已占用周三 19:00–20:00，小组排程不得重复占用该学生的这一时段，其他成员只能看到该时段不可用。

并发变更：AI 生成计划后，成员修改工时，再确认原计划时必须提示重新计算，不能覆盖新工时。

3.5 可用性与质量目标

以下目标在第 2 周需求评审后冻结，最终报告填写实测结果与未达标原因。

维度	初步验收目标
学习成本	5 名未参与开发的学生中至少 4 名，能在简短引导后 10 分钟内完成作业录入、信息确认并获得计划草案
任务可见性	从首页最多 3 次主要操作找到“我今天要做什么、为何被阻塞、何时到期”
要求抽取	对 20 份与调试样例分离的脱敏或模拟说明，由两人核对标注；截止日期和交付物字段的精确率、召回率目标均不低于 90%
排程正确性	至少 25 个可行与不可行案例；系统接受的计划中硬约束违反数为 0，并正确提示预设不可行情形
业务性能	在 100 个测试账号、20 个并发业务请求下，普通读写 API 的 P95 响应时间小于 2 秒；单独统计 AI 延迟
AI 可恢复性	AI 请求展示运行状态，60 秒未完成时进入超时或可重试状态；手工编辑、看板和既有计划仍可使用
权限与追溯	预设跨空间访问测试全部拒绝；每次已生效计划可追溯到确认人、来源和版本
界面可访问性	状态同时使用文字与颜色；关键表单支持键盘操作；移动端可查看任务、评论和确认进度

3.6 后续系统设计的主要对象

领域对象	关键关系与责任
用户、空间、成员关系	用户通过成员关系加入空间，角色属于该关系
课程、作业、要求条目	作业属于课程及空间；要求保存原文片段与确认状态
任务、依赖、交付证据	要求可对应多个任务，任务可覆盖多个要求；依赖构成有向无环图
可用时段、计划版本、时间块	排程读取可用时段；已确认版本生成时间块并保留历史
变更草案、智能代理运行记录	保存输入版本、拟修改内容、工具调用结果及确认状态
评论、通知、审计记录	支持协作、提醒及争议追溯

后续建模重点包括用例图、任务状态图、计划确认时序图、领域类图与数据库 ER 图。复杂度来自相互影响的业务约束，以及人、智能代理和排程服务之间的责任划分。

4. 竞品分析、差异化与创新点

4.1 同类产品与场景适配分析

调研时间为 2026 年 10 月 11 日。以下能力描述来自官方页面；“场景局限”是根据产品公开定位作出的适配分析，不表示已经穷尽其全部配置或插件。

产品	已核实能力	针对本课题的场景局限 / 适配成本
Notion	提供项目数据库、任务与子任务、依赖、看板 / 日历、权限以及 AI 项目文档功能。官方项目页面	可配置性强，但要形成“作业原文—要求—任务—证据”的教学流程，仍需要设计专门的数据结构、模板与规则
Motion	按截止日期、优先级和依赖自动安排任务，并在任务变化后重新优化日程。官方任务管理页面	已覆盖智能排程，因此单纯“自动安排时间”不足以构成本项目特色；本方案需进一步解决课程要求追溯与交付检查
MyStudyLife	管理课表、作业和考试；Scout 已能通过对话帮助规划日程和复习计划。官方功能页面	已覆盖学生场景与 AI 规划；本项目重点验证小组任务依赖、计划变更确认和交付证据关联是否带来额外价值

4.2 差异化定位

本项目以课程作业要求为起点，贯通要求核对、任务拆解、成员分工、约束排程、计划变更和交付检查。相较于通用协作工具，课程作业的要求与交付关系被直接纳入数据模型；相较于以个人日程为中心的规划流程，本方案重点处理小组依赖和跨作业的时间协调。

4.3 本项目的创新点与验证方式

创新点	具体设计	对用户的价值	验证方式
作业要求与交付证据的贯通追溯	每条要求保留原文出处，并关联任务与成果链接	既能查看任务为何存在，也能检查是否缺少交付物	对模拟作业预置遗漏项，验证用户能否发现并定位到原要求
个人时间与小组依赖协同规划	将多课程时间占用、成员可用时段和前置任务共同作为排程约束	减少各自计划都合理、合起来却无法执行的情况	用跨课程冲突和前置任务延期案例检查计划可行性及冲突解释
可确认、可追踪的 AI 计划变更	代理生成草案和变更差异；确认时再次校验版本，生效后保留历史	成员知道哪些安排变化、变化依据是什么，以及谁确认了变更	测试临时请假、修改工时和并发编辑，检查提示、确认及版本记录

上述设计属于面向课程作业的场景整合与交互创新，不宣称具有全球独占功能。是否优于配置良好的现有产品，需要通过同任务对照测试验证。

5. 技术架构与技术栈

5.1 初步技术选型

层次	建议选型	选择理由与使用边界
平台	响应式 Web 应用	便于课堂演示和多设备访问；首版优先适配桌面规划及手机查看
前端	TypeScript、React、Vite、Ant Design	实现表单、看板、周视图与计划差异展示，使用一致的组件交互
后端	Python、FastAPI、Pydantic	实现 REST API、权限、业务状态机及结构化参数校验
数据层	PostgreSQL、SQLAlchemy、Alembic	维护关联数据、事务、计划版本及数据库迁移
排程服务	Python 拓扑排序与基于可用时段的启发式分配	先满足硬约束，再改善负载分布；清楚说明未找到可行方案，不声称求得全局最优
智能代理	单代理工具调用循环、显式运行状态、HTTPX	限制可用工具和执行步数，便于调试、回放与评估
模型服务	支持工具调用的 DeepSeek API，封装为可替换适配器	开工时比较可用模型的质量、延迟与费用，选定并记录版本；不绑定会变化的产品型号
后台处理	数据库持久化任务表与独立 Python worker	保存运行状态、重试次数和到期提醒；进程重启后可恢复或标记中断
测试与部署	pytest、Playwright、Docker Compose	验证排程、权限及关键用户路径，统一演示环境

React 适合构建组件化界面，FastAPI 提供基于类型的接口开发与 API 文档能力；结构化模型输出仍需业务端校验。React 文档、FastAPI 文档、Pydantic 文档。DeepSeek API 提供工具调用接口，但文档也指出模型参数可能不符合预期，因此不能将模型输出直接视为合法操作。DeepSeek API 文档

5.2 系统架构与设计取舍

采用模块化单体：身份权限、作业要求、任务协作、排程、计划版本、通知和代理编排分别组织为模块，共用数据库，通过明确的服务接口交互。API 与后台 worker 可以分别运行，但共享业务服务层，避免重复实现权限和状态规则。

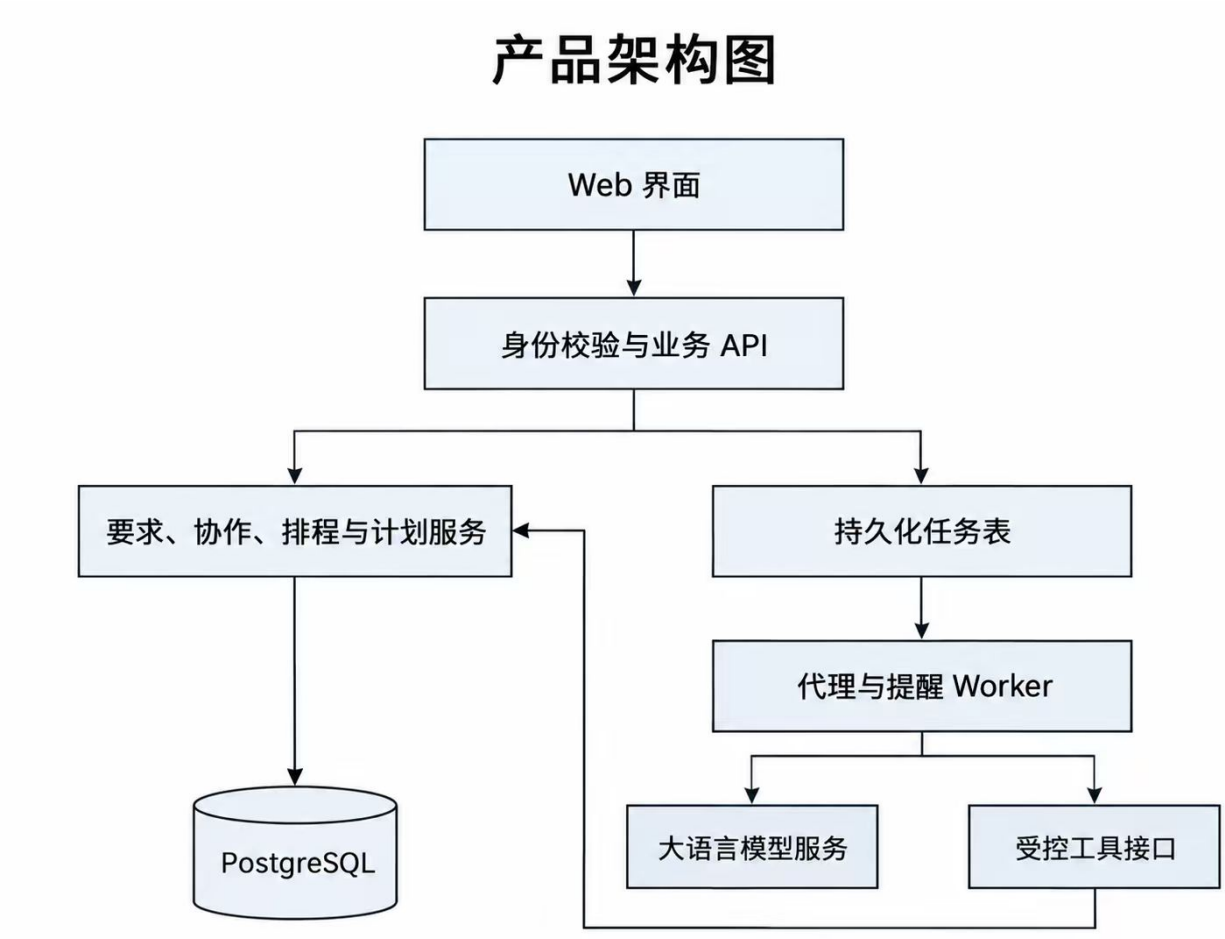


图 2：产品架构

短作业说明按已授权的要求条目检索并提供给模型，首版无需单独向量数据库。排程服务返回结构化可行性与冲突信息，模型负责解释和组织建议。时间统一存储为明确时区下的时间点，界面显示用户时区，默认 Asia/Shanghai。

在应用计划时，对受影响成员的时间分配进行串行化检查，并核对计划版本，防止两个小组同时占用一个成员的同一时段。确认请求具有幂等键，重复点击不会创建两份正式计划。数据库锁和事务机制可作为这一设计的实现基础。PostgreSQL 并发控制文档

5.3 外部依赖与成本控制

首版外部依赖主要是模型 API；站内提醒、排程和任务管理在本地服务中完成。模型密钥仅保存在服务端，按用户设置调用限额，并记录实际用量。日历双向同步、邮件推送和教学平台连接均不是首版前提。

课程原型可先本地部署。建议为所选方案设置不超过 300 元的试验费用上限，优先用于模型评测及必要的短期托管；这是团队预算建议，不是服务商报价。费用达到上限时停止新的收费调用，保留已有数据与手工流程。

6. 计划：团队组织、方法论、工具与排期

6.1 团队组织

成员	主责	配对与共同责任
A：董泽福	需求访谈、产品待办、交互原型及前端整合	与 B 共同核对用户故事、权限及验收标准
B：雍致鸣	后端、数据库、成员权限与计划版本	与 C 共同定义工具接口和并发规则
C：李一芃	作业要求抽取、代理编排与排程算法	与 A、B 共同建设评估数据与异常案例

每项关键模块至少有一名备份维护者。所有成员参与需求分析、设计评审、编码与测试，分工表示主要责任，不意味着只负责单一文档或演示。

6.2 方法论与协作工具

采用借鉴 Scrum 的轻量迭代，每两周一个迭代。A 兼任产品待办负责人，迭代协调工作轮值；每周两次简短同步，迭代末进行演示和回顾。这是适应课程工时的实践安排，不宣称完整执行 Scrum 的所有事件要求。
Scrum Guide

使用 GitHub 仓库、Issues、Projects 和 Pull Request 管理代码与工作项；Markdown 保存需求、架构决策和测试记录；Mermaid 或 draw.io 保存模型图；群聊只用于通知，正式决定进入仓库。每个合并请求至少由另一名成员检查，并通过与变更相关的自动化验证。

6.3 采用智能代理开发方式

产品中的学习规划代理与开发过程中使用的编码代理分别设计、分别评价。 即使某次演示关闭产品 AI，团队仍按课程要求使用智能代理辅助系统设计、设计优化和实现。

开发流程为：成员撰写需求编号、接口契约和验收案例；编码代理提出方案并实现受限工作项；代理运行测试并提交结果；成员核对设计、数据影响和测试覆盖后评审合并。保留任务说明、关键交互摘要、代码差异、测试结果和人工修改理由，不把自动生成内容直接视为已通过评审。

代理适合辅助生成接口骨架、候选测试、文档和重构建议；排程约束、权限边界和数据迁移由团队共同审查。迭代回顾记录代理节省了哪些工作、引入了哪些错误以及后续如何改进任务描述。

6.4 初步排期

周次	主要工作	可检查交付物
第 1–2 周	访谈、竞品体验、界定范围、建立基线	访谈纪要、角色画像、问题陈述、P0 待办与需求规格初稿
第 3–4 周	用例、领域建模、原型与架构；建立基础工程	用例图、关键活动图、ER 图、接口契约、架构决策记录；可运行的账号与作业流程
第 5–6 周	任务、依赖、可用时段及确定性排程	手工端到端流程、排程测试集、冲突说明界面
第 7–8 周	要求抽取、工具调用、计划草案与确认	AI 端到端流程、计划版本、重试与过期草案处理
第 9–10 周	集成、可用性测试、并发权限验证、设计优化	需求追踪矩阵、测试报告、优化前后对照；冻结 P0 范围
第 11–12 周	缺陷修复、文档完善、演示与答辩准备	可部署版本、系统设计说明、用户手册、演示数据和项目复盘

首版未通过验收时不启动 P1。最终追踪矩阵应能从 A-F01 等需求编号定位到相关用例、模块、接口和测试，避免设计图与实现脱节。

7. 挑战：技术风险与组织风险

风险	概率 / 影响	应对方式
AI 漏掉要求或补出原文没有的信息	高 / 高	保存原文出处，区分明确要求与建议，人工确认；用固定评估集追踪错误
工时估计偏差使计划缺乏可执行性	高 / 中	允许学生修改，记录实际耗时；保留可配置缓冲，不承诺估计等同事实
多课程、多成员约束导致无可行排程	中 / 高	先实现规模有限的启发式方案；显示冲突和容量缺口；保留手工调整入口
并发修改造成计划覆盖或重复占用	中 / 高	版本校验、事务内冲突检查、幂等确认及针对性并发测试
私人课程或时间安排被其他小组看到	中 / 高	空间隔离，只输出必要忙闲信息；代理查询也经过同一权限服务
API 延迟、不可用或费用超限	中 / 中	限时、有限重试、用量配额；AI 与手工业务解耦
组员任务不均或关键模块只有一人理解	中 / 高	每周查看实际投入，关键模块配对开发，轮换评审与演示
为追求 AI 效果不断扩大需求	高 / 高	第 2 周明确 P0，新增需求需说明替换掉的工作；优先完成可验收流程

8. 职业成长：这个项目对我个人的价值

8.1 共同成长目标与个人陈述

以下为团队预期成长目标：

“通过这个项目，我希望把自己在课程作业中遇到的协调问题转化为有证据、有边界、可以验证的软件需求。我将学习如何访谈用户、建立用例和领域模型，并把需求追踪到接口与验收测试，而不仅仅是做出一个能运行的页面。

在技术能力上，我希望掌握多人协作中的权限、状态管理、并发更新和异常恢复，并理解智能代理如何调用业务工具以及如何受到系统规则约束。在团队协作上，我希望提高拆分工作、评审设计、处理分歧和解释技术取舍的能力。

最终，我会形成包含需求规格、模型图、架构决策、运行原型和测试报告的作品集，用于展示自己分析复杂问题与完成团队项目的能力。”

个人成长证据包括：主导的访谈或设计记录、至少一项架构取舍、负责模块的代码与测试、一次用户反馈引发的改进，以及对编码代理输出进行纠错的实例。

8.2 各成员的预期收获

下表按暂定职责列出个人成长重点，属于预期目标；项目结束后由各成员结合实际工作和证据撰写个人总结。

成员	预期个人收获	可用于说明成长的成果
A：董泽福	学会从访谈中识别需求、安排优先级，将业务问题转化为可用的界面与交互	访谈纪要、用户故事、原型及根据用户反馈完成的界面改进
B：雍致鸣	掌握领域建模、关系数据库设计、权限控制与并发更新，能够解释一致性设计	ER 图、接口契约、计划版本实现及跨空间访问与并发测试
C：李一芃	学会设计有约束的代理工具调用、排程算法和 AI 评估，区分模型建议与确定性规则	工具定义、排程模块、要求抽取评估集与错误案例分析

所有成员还将练习代码评审、迭代协作、技术表达和对编码代理输出进行验证的能力；最终个人陈述应使用本人实际参与的工作，避免三位成员提交相同内容。

9. AI / Agent 角色与参考资料

9.1 智能代理的角色

代理承担“理解请求—读取授权上下文—调用业务工具—组织建议—等待确认”的流程。它可根据问题选择工具并利用返回结果继续处理，因此不仅是生成一段建议文字的聊天界面。

工具示例	功能	权限与写入边界
get_assignment_requirements	获取已确认要求及原文出处	只读取用户有权访问的作业
get_team_availability	获取排程所需的可用时段	对其他成员只提供必要忙闲信息

工具示例	功能	权限与写入边界
validate_task_dependencies	检查依赖、工时和任务字段	不修改正式任务
simulate_schedule	计算安排和冲突	返回模拟结果，不占用正式时间块
create_plan_proposal	保存待确认的计划草案	记录依据版本，不直接生效

正式应用计划由用户确认接口触发，该接口重新验证权限、版本及所有约束。代理没有直接修改数据库或绕过确认的工具。每次运行设置工具步数和重试上限，并保存工具名称、参数摘要、结果与最终状态，支持问题定位。

作业原文被当作业务数据，不能通过其中夹带的指令扩大工具权限。首版评估分别衡量要求抽取、工具选择、参数合法性、计划约束满足情况与用户采纳情况；不能只用“回答看起来流畅”判断成功。

9.2 参考资料及用途

Notion Projects：项目协作和任务依赖的竞品参考。

Motion AI Task Manager：智能排程和动态调整的竞品参考。

MyStudyLife Features：学生课表、作业和 AI 规划的竞品参考。

FastAPI Documentation：接口设计、类型和服务组织。

PostgreSQL Explicit Locking：多人并发计划确认的实现参考。

DeepSeek Chat Completions API：工具调用与参数校验。

Playwright Documentation：关键用户流程的端到端验证。

The Scrum Guide：迭代、检视与适应的实践参考。

10. AI 工具使用反思

报告里用到了 AI 工具，使用情况如下：

10.1 工具链与任务

我们主要用 ChatGPT 来帮忙：搭方案的整体框架、起草需求和解决方案的初稿、整理技术选型、润色文字，还用它生成了报告里的业务流程图和产品架构图；竞品分析和业务规则这些内容是组里讨论后自己补的

10.2 成本与人工投入

用的是 ChatGPT Plus 订阅，大概投入了一共4小时左右做讨论、核对和补充

10.3 优点与局限

好处是效率高、结构完整，能很快把想法组织成规范的章节，生成的流程图也能直接用；不足是像排程约束、并发规则这些专业内容得靠我们自己改，生成图偶尔细节不对，也要人工检查

10.4 验证方式

我们核对的方法是：把 AI 写的内容对照课程作业的要求检查，竞品的描述都去官方页面核实过，流程图也跟业务规则逐项比对过；报告里的验收数字都标明了是目标值，不是实测结果

10.5 人工增强

在 AI 初稿的基础上，我们补了6条业务规则、验收案例、P0/P1范围边界、风险应对和 12 周排期，还改了 AI 写得不准确的地方，统一了术语

10.6 留存材料

留存的材料有：对话记录、AI 生成的流程图和架构图，还有一份对照课程要求做的核对清单

补充披露

本次使用 Codex 整合材料、修正名称及三人团队表述，并通过 Python 排版生成 PDF。材料未记录具体订阅金额、Token 用量、API 费用或免费额度情况；历史提示词原文与核对清单未随附件提供，未作虚构补充。已留存的对话记录与核对清单可作为配套材料提交。